

Temporal window framing in MasterHub

Research and implementation guide for manual and BCI mental commands

Prepared for Noushith and a new MasterHub builder

Research date 25 September 2026

MasterHub should treat temporal framing as a command interpretation layer between input recognition and device execution. It gives a first gesture time to become an ordered two gesture command, provides a visible cancellation interval, and prevents a stream of repeated BCI predictions from becoming repeated actions. It does not itself recognize thoughts or prove that a detected command was intentional.

The existing project provides a useful prototype, but its manual and live BCI paths have different timing rules. For a new build, use one server owned framing state machine, with separate manual and BCI input adapters, explicit control ownership, fixed deadlines, and device acknowledgements. Start with simulated actions and a low risk output such as a test LED before connecting movement hardware.

This guide explains the current implementation, the underlying research concepts, a proposed architecture, implementation pseudocode, calibration, and acceptance tests. Findings marked Current behavior describe the inspected source snapshot. Proposed design and example parameter values are engineering recommendations, not experimentally validated results.

Reading order

Sections 1 to 3 explain the terminology and existing behavior. Sections 4 to 8 specify the new implementation. Sections 9 to 12 cover calibration, verification, delivery, and troubleshooting. Section 13 lists web references and repository evidence so a builder can check the claims independently.

1 What a temporal window actually means

Four different kinds of time window

A raw EEG analysis window is a block of voltage samples used to calculate features or classify activity. For example, a hypothetical two second segment sampled at 128 Hz contains 256 samples per channel. A sliding analysis window can overlap its predecessor. Its size and stride belong to the signal processing model, not the MasterHub gesture grammar.

A prediction qualification interval asks whether a recognized action is sufficiently stable to be accepted. This may use dwell time, a majority vote, hysteresis, or accumulated evidence. A longer interval can suppress isolated predictions but also increases latency. MasterHub's current live controller has zero additional hold time: one qualifying active sample can start the command frame. [R1]

A command composition window starts when the application accepts the first gesture. During this fixed interval it may accept a distinct second gesture and interpret the ordered pair. This is the temporal window implemented by the current MasterHub controller. Its default is 4 seconds, with a configurable range of 2 to 10 seconds. [R1, R2]

A release interval or refractory interval determines when another command is permitted. Release asks for a return to neutral; refractory time simply suppresses new activations for a duration. These are different mechanisms and should have different settings. A stream freshness timeout is another separate timer: it checks whether input is still arriving.

The deadline rule

Let t_0 be the server monotonic time when the first gesture is accepted, W the composition duration, and $D = t_0 + W$ the deadline. A second gesture belongs to the same frame only if it is accepted before D . Capturing it must not restart or extend D . At or after D , the old frame is finalized before processing further gesture input.

For a fixed deadline implementation, the additional composition latency is approximately W , plus scheduler delay and dispatch overhead. Overall user latency also contains intention formation, headset classification, optional qualification dwell, network transit, and device execution. A fast HTTP response does not mean the user experienced a fast command.

Use a monotonic clock for durations because wall clock corrections can move calendar time. Record UTC timestamps separately for human readable logs. A monotonic timestamp from one process or machine is not a portable timestamp that another machine can subtract from its own clock. [S6]

Why a delay alone is insufficient

If one false push starts a frame and subsequent neutral samples merely allow it to finish, a four second delay still ends in a false push action. The delay provides time to combine or cancel; it is not continuous evidence accumulation. A new implementation must choose explicitly whether to latch an accepted discrete action or require continuing evidence. Motion commands need a separate continuing intent policy.

2 How the current MasterHub works

Manual action selection

In `static/masterhub.js`, `startTemporalFraming` stores a payload, target, label, and source. It sets a deadline with `performance.now()` and updates the display approximately every 40 ms. When time expires, `executePendingImmediately` sends the saved action. The UI also exposes cancellation and an Execute Now path. A selected action is already a resolved command, so this flow is a preview delay rather than necessarily a two gesture parser. [R3]

Manual arrow gestures

Up maps to push, Down to pull, Left to left, and Right to right. In the first UI phase an arrow selects a domain; in the second it selects a device. The phase three path feeds a gesture into `triggerTemporalGesture`. These phase dependent branches mean that not every arrow press traverses the same framing function. [R3]

In `triggerTemporalGesture`, a first gesture opens a frame. A second gesture forms an ordered pair and dispatches immediately, cancelling the single gesture timer. The keyboard handler also recognizes a two key chord and dispatches that pair immediately. Therefore a manual pair can execute before the configured window ends. A single gesture waits until expiry unless Execute Now is used.

The pair is ordered, not a set. `push+right` and `right+push` can mean different things. A physical chord is especially ambiguous because its meaning depends on observed key arrival order. A new UI should document this or use explicit sequential entry.

The browser implementation checks whether a pending gesture exists, but does not compare its saved deadline before constructing a pair in that branch. A delayed browser timer could therefore leave a pending frame after its intended deadline. The keyboard handler also lacks an explicit `event.repeat` guard in the inspected code. These are reasons to avoid copying the browser logic as the authoritative parser. [R3]

Direct API commands

POST /api/command normalizes a command, gesture, or eeg_window and invokes the engine. That endpoint does not itself impose the temporal window. Setting the timing configuration does not guarantee that every client or HTTP request will be delayed. A new design that requires framing must enforce it on the server and restrict any immediate dispatch endpoint. [R4]

Live BCI command flow

The app owned Cortex worker receives a com sample, maps its action and power, publishes live status, and puts the latest sample into a queue of size one. The worker uses CortexControl rather than sending each prediction through a browser timer. This live path shares the application's engine and mode state. The standalone run_cortex.py entry point is a diagnostic route, not a second controller sharing the app's in memory control state. [R1, R5]

Control must be enabled and the connection, authorization, profile, and recent predictions must be valid. The dashboard considers the stream recent while its latest local receive time is less than 3 seconds old. The processing worker rejects queued samples older than 1 second using a monotonic receive timestamp. These are application safeguards, not guarantees about the age of the underlying EEG measurement. [R5, R6]

One neutral sample arms the current controller because release_seconds is 0. A finite power value in the range 0 to 1 must meet DEFAULT_CONFIDENCE_THRESHOLD. The source default is 0.20, with environment overrides; the actual deployment setting may differ. hold_seconds is also 0. Thus the first accepted active sample starts framing without an extra dwell period. [R1, R7]

The controller snapshots the current mode, target, control generation, duration, and deadline. A distinct qualifying second gesture creates a pair only when that pair is assigned in the current context. It retains the original deadline and stores the smaller of the two accepted powers. Repeated samples of the first gesture do not create repeated commands. Neutral between first and second is optional. After a pair has been captured, further active gestures do not extend that pair. [R1]

An unassigned second gesture cancels the frame rather than falling back to the first action. A second gesture below threshold is ignored, leaving the first candidate pending. At or after the deadline, the next processed sample triggers finalization. There is no independent deadline scheduler in CortexControl: actual dispatch is sample driven, so it may occur later than the displayed zero. [R1, R8]

Before dispatch, the controller checks that control generation, mode, and target still match. A change invalidates the frame. Required parameters must already be saved. They are read at dispatch in the current implementation, rather than being frozen when the frame opens. After finalization, another neutral sample is required to arm again. [R1]

Motion has additional rules

Car and chair direction commands require the deadline triggering sample to match the framed direction and meet the power threshold. If that sample is neutral or another direction, motion is cancelled. Once motion has begun, neutral requests a stop. The worker also attempts to stop motion when control becomes unavailable or a connection is reestablished. [R1, R5]

These software checks should not be mistaken for a hardware dead man switch. A blocked handler, process crash, unavailable transport, or failed stop acknowledgement can defeat a host only stop. A new moving device needs an independent local expiry mechanism and physical stop input.

Documentation discrepancy

docs/CORTEX_MENTAL_CONTROL.md describes a 0.25 second neutral interval and 0.35 second active hold. The inspected controller currently uses zero for both, and tests explicitly exercise immediate frame start and fixed deadline pairs. For reproducing this snapshot, follow the code and its timing tests; update written instructions when policy changes. [R1, R8, R9]

3 Worked timing examples

BCI single action at four seconds

At time 0.00, neutral arms the controller. At 0.20, push with power 0.45 starts a frame in IDLE. Its deadline is 4.20. Further push samples do not dispatch. If a valid incoming sample is processed at 4.24, the frame can resolve to mode_desktop then. A neutral sample at expiry is sufficient for a nonmotion single action. The extra 0.04 seconds is an illustrative sample or scheduling delay, not a measured result. [R1, R10]

BCI ordered combination

In LIGHT_MODE, push at time 0 starts a four second frame. right at 1.00 changes the candidate to push+right. The controller waits until the original 4.00 deadline and then returns to the parent menu on the next processed sample. The deadline is not changed to 5.00. A neutral sample between push and right is permitted but not required. [R1, R8, R10]

The manual contrast

In the manual gesture path, push at 0.00 followed by right at 1.00 dispatches the pair at approximately 1.00, subject to browser and request delays. It does not wait until 4.00. A UI action selected directly can instead use the full preview countdown, with Execute Now available. These are separate current behaviors, not evidence of one universal timing contract. [R3]

Invalid and boundary inputs

In IDLE, a valid push followed by a qualifying pull before expiry cancels if push+pull is unassigned. A pull below the threshold does not form a pair; the original push may still execute. A second gesture processed exactly at the BCI deadline cannot extend the pair, because expiry is checked first. A frame whose timing setting changes from 4 to 8 seconds keeps its original four second deadline; a later frame uses eight. [R1, R8]

Why navigation depth matters

If a workflow takes three framed selections with $W = 4$ seconds, framing alone contributes about 12 seconds. Neutral rearming, thought recognition, and execution add further time. This arithmetic is a planning example, not a benchmark. Reducing unnecessary menu depth can improve task speed without making the classifier more permissive.

4 Research foundations and practical interpretation

BCI predictions are not raw thought commands

EMOTIV's com stream reports a recognized action and a power value. The SDK documentation identifies these as act and pow; a payload can look like {"com":["push",0.62],"sid":"session","time":1234567890.0}. Although MasterHub names the value confidence, it should not be presented as a calibrated probability that the user's intention is correct. A power of 0.62 does not establish 62 percent accuracy. [S1]

EMOTIV documents com output at 8 Hz, giving a nominal interval of 125 ms. This is the output prediction cadence, not the raw EEG sampling rate or a promise of end to end latency. The documentation separately describes EEG streams and their sample rates. Measure actual receive gaps and jitter on the selected setup. [S2]

Mental command detection requires a loaded trained profile. EMOTIV's training workflow uses system events, followed by accepting or rejecting training and saving the profile. Neutral training supplies a reference state. Train and evaluate the intended user rather than assuming a friend's trained profile transfers reliably. [S3, S4]

Asynchronous operation and the idle problem

An always listening BCI must distinguish intentional activity from the much larger amount of idle time. Wu, Li, and Wu's research on asynchronous motor imagery separates resting state prescreening from subsequent task classification. That supports treating intent detection and action identity as separate design problems. It does not validate MasterHub's threshold, prove the proprietary Cortex classifier uses the same method, or prescribe a four second composition window. [S5]

Use the research as a design principle: include substantial neutral and ordinary activity data in evaluation; do not measure only cued commands. A classifier with good accuracy on selected trials can still cause unacceptable unintended actions when left listening for a long period.

Choosing a qualification strategy

For a minimal reproduction, preserve one qualifying sample followed by the fixed command frame. For a more conservative new prototype, compare an additional short dwell, a time weighted vote, and a hysteresis gate. These are proposed alternatives to test, not established optimum settings.

A dwell requires the candidate to remain qualified for a minimum duration and resets on contradictory evidence or excessive gaps. A time weighted vote measures how much recent valid time supports each action; it must include neutral and conflicting evidence rather than discarding them. Hysteresis uses a higher threshold to enter an active state and a lower one to remain active, reducing rapid on off switching.

Overlapping EEG windows or consecutive model predictions may be strongly correlated. Three successive votes are not necessarily three independent confirmations. Test false activations with recorded streams and live neutral sessions rather than estimating reliability by multiplying per sample probabilities.

A custom raw EEG alternative

For a new project using a different headset, acquisition software such as BrainFlow can provide board specific sample rates and signal processing interfaces. It is not a ready made mental command recognizer. A custom route still needs artifact handling, labeled training data, feature extraction, a classifier, an idle detector, and online validation. [S7]

Split training and evaluation by session or recording before creating overlapping windows, so nearly identical samples do not leak across train and test sets. Tune preprocessing on training data only. Keep the downstream command framing layer independent of which classifier supplies labels.

The existing services/eeg_service.py uses a stub classifier rather than a trained EEG model. Sending an eeg_window to that endpoint should therefore not be treated as evidence that the repository already provides a working raw EEG BCI. [R11]

5 Architecture for a new MasterHub

Minimum build components

Use one host computer for the user interface, command server, and initial headset adapter. Add a supported EEG headset with the manufacturer's runtime and a trained profile if using Cortex. For the first output use an on screen simulator or microcontroller LED. Add USB serial or a network transport only after the logical controller passes replay tests. Check current headset and stream access requirements against the vendor documentation before purchasing hardware. [S8]

Suggested software components are a web UI, an input adapter layer, one event queue, a deterministic frame controller, a mode and mapping registry, a command validator, an execution queue, device adapters, and structured logs. The frame controller owns all timing. The browser renders state and submits intent events.

The logical flow is Input source -> normalized event -> qualification and ownership -> frame controller -> context and parameter validation -> command executor -> device acknowledgement. Status events return from each stage to the UI. Maintain one authoritative controller per user and controlled target, with explicit arbitration when targets share resources.

Event contract

Each event should carry an event_id, source, source_session_id, sequence number where available, action, power when applicable, local receive time, and quality metadata. The server adds received_monotonic and receives only allowlisted actions. Keep source timestamps for diagnosis, but do not trust an unverified client clock to enforce a deadline.

A proposed manual event is

`{"event_id":"m17","source":"manual","kind":"gesture","action":"push","context_version":12}`. A proposed BCI adapter event adds source_session_id, sequence, power, and quality. These are design examples; they are not existing repository endpoint schemas.

The pending frame contains frame_id, owner, context_version, mode, target, configuration version, mapping version, first action, optional second action, opened time, deadline, immutable parameter snapshot, and execution status. Freeze parameters at creation or cancel and rebuild the frame if the user edits them.

Ownership and context

Use explicit Manual and BCI control modes. A change of owner cancels pending work and requires BCI neutral rearming. Manual emergency stop overrides either mode. Never let a manual push silently combine with a BCI right merely because they arrive inside the same interval.

Increment context_version on mode, target, mapping, or control ownership changes. Before execution, require the frame's version to match the current version. Merely storing a target label in the UI is insufficient if the server routes against another active device.

One authority across processes

For a first prototype, use one application process with one controller instance and one Cortex worker. Multiple web server workers each holding independent global state can duplicate execution or disagree about arming. A scaled deployment should assign controller ownership to a dedicated service and use shared durable command state and a message channel. Do not solve that by creating a controller in every HTTP worker.

6 The proposed framing state machine

States and transitions

DISABLED accepts status updates and stop requests but cannot open a frame. Enabling BCI control enters WAIT_NEUTRAL. A qualified neutral event enters READY. Manual control can enter READY after an explicit enable action and release of held keys.

READY accepts the first qualified gesture and opens COLLECTING with a fixed deadline. A mapped distinct second gesture changes the stored candidate to a pair and enters PAIR_CAPTURED. Repeated first gesture samples do not produce extra actions. An invalid pair cancels the frame and returns BCI control to WAIT_NEUTRAL.

At expiry, COLLECTING or PAIR_CAPTURED enters VALIDATING. The controller checks freshness, ownership, context, required parameters, mapping validity, and any device specific interlock. It then atomically marks the frame consumed and enqueues one execution. Successful completion enters WAIT_NEUTRAL for BCI, or waits for manual key release before becoming READY again.

Cancel, stop, lost control, profile changes, and target changes invalidate pending work. Stop also bypasses the ordinary queue and invokes the device stop channel. In the proposed fixed deadline design, both singles and pairs wait for the same deadline; this deliberately removes the current manual versus BCI timing mismatch.

Discrete actions versus continuous motion

A discrete action such as turning a lamp on can latch after qualification and wait for expiry. Neutral then means the user has released the thought, not necessarily cancelled the pending action. Provide a separate visible Cancel command. This matches the useful part of current nonmotion behavior.

Motion must be maintained by fresh qualified direction evidence and a short device lease. Expiry of the lease stops movement locally. Renew only while ownership, stream quality, and intended direction remain valid. Do not renew merely because the socket is connected. Neutral, control loss, or manual stop invalidates the lease immediately.

Do not derive the motion stop timeout from the 2 to 10 second composition window. The lease duration needs a separate engineering assessment based on speed, stopping distance, communication jitter, and actuator behavior. Host UI polling is not the safety mechanism.

Pseudocode for the single owner controller

The following is an algorithm sketch, not a complete production module. process_event and tick run on the same serialized event loop. validate and enqueue must preserve the ownership check through command handoff; the executor must not block the framing loop.

```
on event(e):
    now = monotonic()
    reject duplicate, malformed, wrong session or unauthorized e
    if e is STOP:
        cancel_pending(); invalidate_motion_lease(); priority_stop()
        return
    expire_if_due(now) # deadline wins over a second gesture
    if context changed: cancel_pending(); require_rearm()
    update_live_evidence(e, now)
    if BCI and not enabled_and_fresh(): disable(); return
    if e is neutral:
        stop_motion_if_active()
        if no pending and neutral_is_qualified(): state = READY
        return
    if not qualified(e): return
    if state == READY:
        if e.action is mapped or is a valid pair prefix:
```

```

        pending = snapshot(e, deadline=now + configured_w)
        state = COLLECTING
    elif state == COLLECTING and e.action != pending.first:
        pair = ordered_pair(pending.first, e.action)
        if mapped(pair, pending.context):
            pending.pair = pair; state = PAIR_CAPTURED
        else: cancel_pending(); require_rearm()
on periodic tick:
    check_stream_and_device_lease_expiry()
    expire_if_due(monotonic())
expire_if_due(now):
    if no pending or now < pending.deadline: return
    frame = atomically_remove_pending()
    require_rearm()
    if not context_owner_and_quality_valid(frame): reject(frame)
    elif motion(frame) and not fresh_matching_direction(): reject(frame)
    elif missing_parameters(frame): reject(frame)
    else: enqueue_once(frame.frame_id, resolve(frame))

```

An independent scheduler removes dependence on the next BCI packet for ordinary deadline completion. At each tick, freshness still has to be checked; a timer must never cause stale intent to execute after a disconnected stream. Establish and test a tie rule for events arriving exactly at the deadline. The sketch uses processing time and finalizes the old frame first.

7 Mapping and API design

Context sensitive mappings

Preserve a small vocabulary such as push, pull, left, and right, then interpret it within a mode. In the inspected map, IDLE uses push for desktop, pull for embedded mobility, left for IoT, and right for media. Global navigation includes push+right for Back and push+left for the main menu. Media modes can assign right+push to volume up, showing why order matters. [R10]

Validate mappings when the application starts. Every referenced command must exist, every mode transition must be valid, every pair prefix must be recognized, and no reserved stop event may be reassigned. Restrict a frame to at most two gestures initially. Supporting longer sequences requires a grammar and a clearly different interaction design.

Proposed endpoint responsibilities

POST /input-events submits intent to the framing controller and returns an event receipt. GET /control-state returns the owner, mode, target, pending frame, remaining time, freshness state, and last execution result. POST /control enables, disables, or transfers ownership. POST /frames/{id}/cancel cancels only the specified frame. POST /stop uses priority stop handling.

POST /settings/temporal-window validates a finite number and bounds before saving a versioned configuration. GET returns its effective value and units. Reject NaN, infinity, booleans, and invalid JSON types explicitly. The inspected metrics setter checks bounds but lacks an explicit finiteness check, so a new implementation should not copy its validation unchanged. [R2]

Keep the execution API internal where possible. If a trusted manual bypass is required, give it separate authorization and explicit UI wording. A client supplied source value of bci must not grant BCI trust or bypass permission checks.

Execution and acknowledgement

Use frame_id or a derived command_id as an idempotency key. Record a command as queued, sent, acknowledged, failed, or unknown. An HTTP success from MasterHub may indicate acceptance by a handler, not physical completion by a device. Display device acknowledgement separately when available.

Retries of a lamp state command can often be made idempotent by saying `set_on` instead of `toggle`. A movement, email send, or other nonidempotent command requires stronger deduplication or an explicit unknown outcome after transport loss. Exactly once physical execution cannot be promised by an in memory consumed flag alone.

UI behavior

Show the selected device, source, first gesture, captured pair, remaining time, and cancellation control. Explain when neutral rearms and when it stops motion. Present power as detector power rather than accuracy. Show rejected input reasons such as unassigned pair, low power, stale stream, context changed, and missing parameters.

The UI should display a server supplied remaining duration and use a local monotonic clock only to animate between updates. After a reconnect, fetch authoritative state. Do not replay a cached countdown or resubmit an old command automatically.

8 BCI integration procedure

Build the manual path first

Create the command map and implement the serialized state machine with a fake clock. Attach a manual adapter that emits one gesture per deliberate key press and suppresses auto repeat. A corresponding key release is required before another press of the same key is accepted. Route direct button actions through the same controller, or explicitly designate them as privileged immediate actions.

Use a simulated lamp and write logs containing frame start, pair capture, cancel, deadline, resolution, dispatch, and result. Confirm that changing a setting affects only the next frame. Verify stop while idle, collecting, executing, and disconnected.

Connect Cortex

Register the application and keep client credentials on the host service rather than in browser JavaScript. Follow the vendor flow for user access, authorization, headset discovery and connection, session creation, profile loading, and com subscription. API stream availability depends on hardware and access level, so query and handle errors rather than hard coding licensing assumptions. [S3, S8, S9]

Begin in monitor only mode. Check that expected actions and neutral arrive, the profile is correct, and contact quality is usable. Enable actuation only after the user deliberately arms control. On reconnection, clear the frame, invalidate the old session, and require enable plus neutral again.

Normalize and qualify

Reject unknown actions, malformed payloads, nonfinite power, incorrect session identity, and duplicate events. Record local monotonic arrival time. Preserve vendor timestamp separately and monitor unusual timestamp gaps where available. Avoid silently turning missing timestamps into proof of freshness.

Decouple reception from device execution. A latest sample buffer prevents a long queue of old intent, but can miss brief transitions if the consumer is slow. A bounded event queue with sequence tracking and explicit overflow cancellation is preferable when sequence fidelity is essential. Either way, do not allow long device calls to block neutral and stop processing.

Learn the intended user's control patterns

Train neutral and one active action first, then expand only when the user can reliably distinguish them. Use the same deliberate mental strategy during training and operation. Record confusion between actions and ordinary eye, face, and body activity. A temporal parser cannot repair consistently confused input labels.

For a new builder, the Cortex classified command route is the simpler starting point than developing an EEG decoder. The downstream framing protocol can remain the same if a custom classifier is added later.

9 Parameter selection and calibration

Reproduction settings

To reproduce the inspected live controller, start with $W = 4.0$ seconds, allowed W between 2.0 and 10.0, additional hold = 0, release = 0, and source default power threshold = 0.20. These are source defaults, not proof of suitability for a new user. Its stream freshness limit is 3 seconds and queued sample age limit is 1 second. [R1, R2, R5, R6, R7]

Proposed calibration experiment

For a nonmotion simulator, compare composition windows of 2, 4, and 6 seconds. Separately compare qualification dwell of 0, 0.25, and 0.5 seconds, and neutral qualification of 0 and 0.25 seconds. Treat these as initial experimental choices rather than recommendations for physical actuation. Tune power thresholds using the user's observed idle and intentional command distributions, not a universal confidence percentage.

Change one policy at a time initially. A proposed pilot is several short blocks with neutral periods, each single gesture, valid ordered pairs, invalid pairs, cancellation, and menu navigation. Counterbalance parameter order where practical to reduce learning and fatigue effects. Include a separate later session to test whether the chosen setting generalizes.

Record intended action independently from received prediction. Otherwise a log can confirm that software executed its own predicted label without showing whether the user wanted it. With consent, a test operator or a cue log can record intended trials for comparison.

Metrics that answer useful questions

Measure false activations per minute during explicit no control periods. Measure intended commands completed divided by attempted commands, rejected attempts, wrong commands, and cancellations. Record end to end latency from the test cue or declared intention marker to device acknowledgement, and report median and high percentile values as well as the distribution.

Measure second gesture timing relative to the first, so W can be chosen to cover the user's sequence timing without excessive waiting. Track pair confusion separately from single gesture confusion. Include dropped samples, receive gaps, queue age, and stop latency.

Zero false activations in a brief demonstration is weak evidence. Under a simple Poisson model, zero events over T minutes gives an approximate one sided 95 percent upper rate bound of $3/T$ per minute. For example, zero over 30 minutes still permits roughly 0.1 per minute under that model. This is an illustrative statistical estimate, not a validated risk assessment; temporal dependence and changing user behavior weaken the model.

Selecting the final policy

Choose the shortest tested window that supports the user's intended combinations while meeting the agreed false activation and task completion targets. Prefer a simpler menu or fewer active gestures when confusion remains high. Record the selected settings, user profile, hardware, software versions, and testing conditions so another builder can reproduce the result.

10 Verification and acceptance tests

Deterministic framing tests

Use an injected clock rather than sleeping in unit tests. Assert that a single command does not dispatch before D; a pair captured before D dispatches exactly once at D; a second gesture at D cannot join the old frame; repeated active samples do not produce new frames; and unassigned pairs cancel without falling back.

Assert that a below threshold second gesture leaves the original candidate according to the documented policy. Test NaN, infinity, negative power, power above one, empty actions, unknown actions, and missing fields. Test same action twice separately from a held action; the initial proposed grammar rejects repeated action pairs unless explicitly supported.

Assert that neutral rearms only after the required interval, timing edits affect the next frame, parameters are frozen, and changing mode, target, mapping version, owner, profile, or source session cancels the old frame. Assert that duplicate event IDs and command IDs cannot repeat execution.

Concurrency and recovery tests

Deliver manual and BCI events at nearly the same time and confirm ownership prevents mixing. Delay the executor and confirm the frame loop and priority stop remain responsive. Drop the stream at D minus a small interval and ensure stale input cannot execute at D. Drop the network after send but before acknowledgement and verify the result becomes unknown rather than triggering a blind nonidempotent retry.

Test process restart, browser closure, browser reconnect, device reboot, malformed packets, queue overflow, and late acknowledgements. A stale acknowledgement must not be attributed to a newer command with the same action label. Start multiple application workers in a test environment and verify the chosen single controller ownership design.

Motion bench tests

Use a stationary bench setup with drive outputs disconnected or mechanically unloaded. Verify neutral stop, host crash, cable removal, Wi Fi loss, delayed messages, direction changes, and hardware lease expiry. Confirm that physical stop operates without the host. Only progress to movement after measuring actual actuator stopping behavior under the intended operating conditions.

Existing repository test evidence

tests/test_cortex_control.py contains explicit tests for fixed deadline pairs at windows of 2, 4, and 8 seconds, invalid pair cancellation, control pause cancellation, next frame configuration changes, and low power second gesture handling. These tests document intended behavior. This research report is based on source inspection; it does not claim a fresh automated test run or a hardware validation. [R8]

11 Step by step delivery plan

Milestone one deterministic simulator

Implement mappings, the event contract, a fake clock, and the frame controller. Deliver a simulator that shows singles, ordered pairs, cancellation, context changes, and stop. Completion means all deterministic timing tests pass and no physical output is connected.

Milestone two manual end to end control

Connect the UI and a harmless output through the actual transport. Deliver event and command IDs, acknowledgement display, configuration validation, and ownership. Completion means retries and browser reconnection cannot repeat an old action.

Milestone three BCI monitor and replay

Integrate Cortex in monitor mode, record normalized predictions with consent, and replay representative streams into the same controller. Deliver stream age monitoring, session checks, and clear qualification feedback. Completion means a disconnected or stale stream cannot open or complete an executable frame.

Milestone four supervised BCI actions

Enable the harmless output with the intended user. Run the calibration protocol, document selected settings, and compare manual and BCI behavior against the same timing contract. Completion means agreed task and false activation criteria are met on held out sessions.

Milestone five device specific hardening

Add each real device as a separate adapter with acknowledgement semantics, timeouts, parameter validation, and local interlocks. For mobility, add leases and independent stop before movement trials. Completion means failure tests have recorded outcomes, not just successful demonstrations.

Handoff package for the friend

Provide the event schema, command map, state transition specification, settings with units, simulator, test suite, installation instructions, trained profile procedure, and device protocol. Include a known limitations list and a versioned change log. Keep credentials and personal EEG recordings out of the shared repository.

12 Troubleshooting guide

If every held thought triggers repeated commands, inspect neutral rearming, event deduplication, and whether multiple controllers are running. If a pair triggers its first action too early, check whether manual immediate pair behavior or an immediate API bypass is still active.

If the countdown reaches zero without execution, inspect stream freshness, the latest received sample, context mismatch, required parameters, and device acknowledgement. In the current BCI implementation, expiry is evaluated on an incoming processed sample. In the proposed implementation, inspect the independent tick scheduler.

If commands change meaning unexpectedly, log mode, target, owner, and mapping version at frame creation and dispatch. If a pair reverses meaning, inspect actual input order and keyboard chord handling. If an old command executes after reconnection, invalidate sessions and frame IDs and stop replaying buffered input.

If intentional commands are often missed, compare threshold rejection, contact quality, profile selection, classifier confusion, and user timing before increasing W . Increasing W only helps the time allowed for combinations; it does not directly improve EEG decoding.

If motion persists after neutral, inspect stop delivery and acknowledgement, handler blocking, and local device watchdog behavior. Disabling the UI is not evidence that the actuator stopped. If logs show low latency while control feels slow, include framing and intention recognition time in the measurement.

13 References and evidence map

External primary sources

All web sources below were consulted on 25 September 2026. Vendor pages describe current documented interfaces; verify them again when building against a different software version.

[S1] EMOTIV Cortex API, Data sample object. Defines mental command action and power payload fields.

<https://emotiv.gitbook.io/cortex-api/data-subscription/data-sample-object>

[S2] EMOTIV Cortex API, Data Subscription. Documents stream types, profile requirement for meaningful com output, and nominal stream rates. <https://emotiv.gitbook.io/cortex-api/data-subscription>

[S3] EMOTIV Cortex API, BCI. Describes profiles, training events, accept or reject, saving, and loading.

<https://emotiv.gitbook.io/cortex-api/bci>

[S4] EMOTIV EmotivBCI, Training neutral. Explains the neutral reference training state.

<https://emotiv.gitbook.io/emotivbci/mental-commands/training-neutral>

[S5] Huanyu Wu, Siyang Li, and Dongrui Wu. Motor Imagery Classification for Asynchronous EEG Based Brain Computer Interfaces. IEEE Transactions on Neural Systems and Rehabilitation Engineering 32, 527 to 536, 2024. DOI 10.1109/TNSRE.2024.3356916. Author paper record and abstract: <https://arxiv.org/abs/2412.09006>

[S6] Python documentation, time. Describes monotonic clocks and timing functions.

<https://docs.python.org/3/library/time.html>

[S7] BrainFlow documentation, User API. Documents board acquisition and signal processing APIs.

<https://brainflow.readthedocs.io/en/stable/UserAPI.html>

[S8] EMOTIV Cortex API, Getting Started. Describes supported hardware, access categories, and application registration. <https://emotiv.gitbook.io/cortex-api>

[S9] EMOTIV Cortex API, Overview of API flow. Describes Launcher login and application access preparation.

<https://emotiv.gitbook.io/cortex-api/overview-of-api-flow>

Repository evidence

Repository root for this inspection is D:\GALATICX\masterhub_iot_noushith. The following references identify files and symbols in the inspected working tree, rather than an immutable published release. Local code may change after this report.

[R1] cortex/control.py, CortexControl.process, _dispatch, reset, stop_motion, and snapshot. Source for arming, pairing, fixed deadlines, context checks, and motion handling.

[R2] core/metrics.py, MetricsTracker.get_temporal_window and set_temporal_window; api/routes.py, temporal_window_config. Source for default value, permitted range, and configuration endpoint.

[R3] static/masterhub.js, startTemporalFraming, executePendingImmediately, triggerTemporalGesture, dispatchDirectGesture, handleArrowKeyDown, and resolveArrowCombo. Source for browser framing and manual gesture differences.

[R4] api/routes.py, handle_command; core/input_processor.py, InputProcessor.normalize and _resolve_gesture. Source for immediate API routing and normalization precedence.

[R5] cortex/run_live.py, _receive_prediction and LiveRunner.run_forever. Source for app owned processing, latest sample queue, sample age filtering, and loss handling.

[R6] cortex/dashboard.py, update_prediction, control_token, live_snapshot, and set_control. Source for enable conditions and stream freshness.

[R7] simulator/config.py, DEFAULT_CONFIDENCE_THRESHOLD; cortex/prediction_mapper.py, map_mental_command. Source for power default, environment overrides, normalized fields, and range validation.

[R8] tests/test_cortex_control.py. In particular test_headset_combination_waits_until_original_deadline, test_invalid_combination_cancels_without_fallback, test_pause_cancels_captured_combination, test_window_changes_apply_to_next_frame_only, and test_second_gesture_below_threshold_does_not_form_combination.

[R9] docs/CORTEX_MENTAL_CONTROL.md. Older operational timing descriptions; compare with R1 before using its hold and release instructions.

[R10] mappings/gesture_map.json. Source for context dependent single commands, ordered pairs, and navigation mappings.

[R11] services/eeg_service.py, EEGService.classify and _classify_stub. Source for the distinction between the placeholder raw EEG route and vendor classified mental commands.